

DESIGN AND VERIFICATION OF AN 8-BIT COMBINATIONAL ARITHMETIC LOGIC UNIT (ALU)

USING VERILOG HDL

PROJECT REPORT

Prepared by

Dharshan R

Electronics and Communication Engineering

IIIT Kottayam

Tools: Verilog HDL | Icarus Verilog | GTKWave | GitHub

1. Abstract

This project presents the design and functional verification of an 8-bit combinational Arithmetic Logic Unit (ALU) using Verilog HDL. The ALU accepts two 8-bit operands and a 3-bit operation-select input and supports eight arithmetic and bitwise operations: addition, subtraction, AND, OR, XOR, NOT, increment, and decrement. In addition to the 8-bit result, the design generates Carry and Zero status outputs. The RTL design was simulated using Icarus Verilog and its signal behavior was examined using GTKWave. Verification was performed using directed functional tests, boundary/edge-case tests, and 100 randomized test cases. A total of 113 test cases were executed, with all 113 tests passing.

2. Objectives

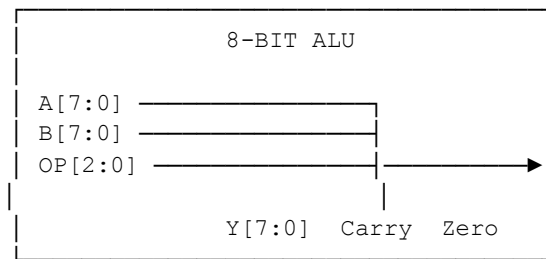
- Understand the fundamentals of Verilog HDL and RTL-based digital design.
- Design an 8-bit combinational ALU using Verilog.
- Implement arithmetic and bitwise operations using a 3-bit operation selector.
- Generate Carry and Zero status flags.
- Develop a dedicated Verilog testbench for functional verification.
- Verify normal operation, boundary conditions, and randomized input combinations.
- Analyze the resulting simulation waveforms using GTKWave.
- Organize the source code, verification files, simulation evidence, and documentation in GitHub.

3. Project Specifications

Parameter	Specification	Description
Operand width	8 bits	Inputs A and B are 8-bit values
Operation select	3 bits	Selects one of eight ALU operations
Result	8 bits	ALU output Y
Status outputs	2	Carry and Zero
Architecture	Combinational	Output depends on present inputs
Verification	113 tests	13 directed + 100 randomized

4. ALU Operations

OP	Operation	Expression	Purpose
000	ADD	$A + B$	Addition
001	SUB	$A - B$	Subtraction
010	AND	$A \& B$	Bitwise AND
011	OR	$A B$	Bitwise OR
100	XOR	$A \wedge B$	Bitwise XOR
101	NOT	$\sim A$	Bitwise complement of A
110	INC	$A + 1$	Increment A
111	DEC	$A - 1$	Decrement A



5. Design Methodology

The ALU was implemented as a combinational RTL module. The module receives A, B, and OP as inputs. A case statement selects the required operation according to OP. The result Y and Carry are assigned for each operation, while the Zero output is generated by checking whether Y is equal to zero.

5.1 RTL Structure

The main design is contained in the alu8 module. Y and Carry are assigned inside an always @(*) combinational block. The expression {Carry, Y} = A + B is used for addition so that the nine'th bit of the addition is captured as Carry while the lower eight bits form Y.

5.2 Status Flags

- Carry: For addition and increment, Carry indicates overflow beyond the 8-bit range. For subtraction and decrement, it indicates the implemented borrow/underflow condition.
- Zero: Zero is asserted when the resulting 8-bit value Y is 00000000.

6. Verification Methodology

A separate testbench was developed to automatically apply inputs to the ALU and compare the observed outputs against expected values. The testbench includes a reusable check_result task and maintains pass and fail counters.

6.1 Directed and Edge-Case Tests

- One or more functional cases for each of the eight operations.
- 255 + 1 to verify addition overflow and Carry.
- 100 - 100 to verify the Zero flag.
- 0 - 1 to verify subtraction underflow/borrow behavior.
- 255 + 1 using increment to verify wrap-around and Carry.
- 0 - 1 using decrement to verify wrap-around and underflow indication.

6.2 Randomized Verification

After the directed tests, the testbench generated 100 randomized combinations of A, B, and OP. For every randomized case, an expected result and expected Carry value were calculated and compared automatically with the ALU outputs.

7. Verification Results

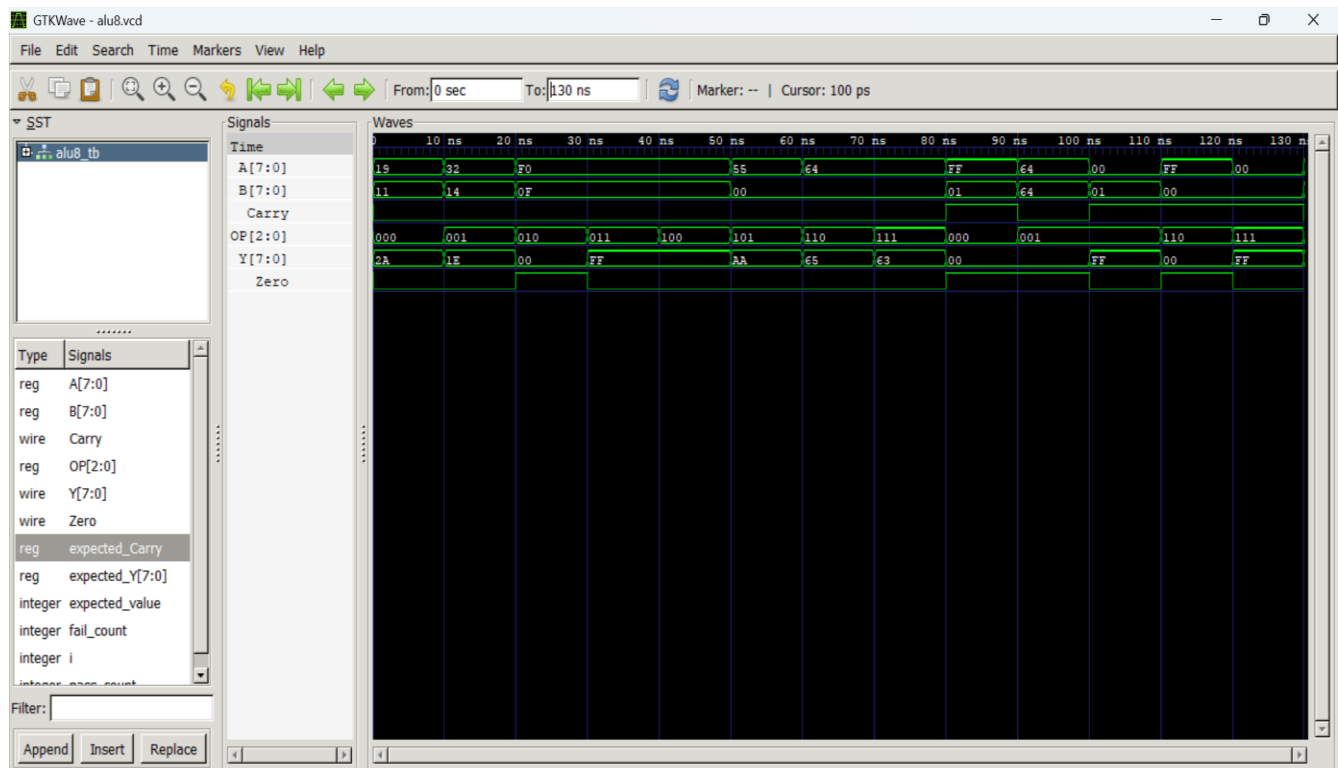
Metric	Result
Directed / edge-case tests	13
Randomized tests	100
Total tests	113
Tests passed	113
Tests failed	0
Pass rate	100%

VERIFICATION RESULT: 113 / 113 TESTS PASSED

8. Simulation and Waveform Evidence

The design was simulated using Icarus Verilog. The generated VCD file was opened in GTKWave to inspect the relationship between the inputs, operation select, result, and status flag

WAVEFORM:



VERIFICATION:

```
PASS: A=50 B=c7 OP=011 -> Y=d7 Carry=0 Zero=0
PASS: A=05 B=77 OP=111 -> Y=04 Carry=0 Zero=0
PASS: A=9b B=3a OP=010 -> Y=1a Carry=0 Zero=0
PASS: A=b5 B=c4 OP=010 -> Y=84 Carry=0 Zero=0
PASS: A=cd B=c9 OP=100 -> Y=04 Carry=0 Zero=0
PASS: A=16 B=a5 OP=011 -> Y=b7 Carry=0 Zero=0
PASS: A=0f B=86 OP=111 -> Y=0e Carry=0 Zero=0
PASS: A=8c B=e8 OP=010 -> Y=88 Carry=0 Zero=0
PASS: A=20 B=1d OP=111 -> Y=1f Carry=0 Zero=0
PASS: A=a9 B=2b OP=001 -> Y=7e Carry=0 Zero=0
PASS: A=71 B=02 OP=101 -> Y=8e Carry=0 Zero=0
PASS: A=6c B=89 OP=010 -> Y=08 Carry=0 Zero=0
PASS: A=1c B=72 OP=010 -> Y=10 Carry=0 Zero=0
PASS: A=95 B=c0 OP=111 -> Y=94 Carry=0 Zero=0
PASS: A=93 B=d0 OP=000 -> Y=63 Carry=1 Zero=0
PASS: A=02 B=ac OP=010 -> Y=00 Carry=0 Zero=1
PASS: A=8a B=09 OP=010 -> Y=08 Carry=0 Zero=0
PASS: A=58 B=aa OP=011 -> Y=fa Carry=0 Zero=0
```

```
=====
      8-BIT ALU SUMMARY
=====
Tests Passed : 113
Tests Failed : 0
ALL TESTS PASSED!
=====
alu8_tb.v:235: $finish called at 1130000 (1ps)
```

9. Tools Used

Tool / Technology	Use
Verilog HDL	RTL design and testbench development
Icarus Verilog	Compilation and simulation
GTKWave	VCD waveform analysis
GitHub	Version control and project presentation

10. Learning Outcomes

- Developed a practical understanding of combinational RTL design in Verilog.
- Learned how operation selection can be implemented using a case statement.
- Understood the use of Carry and Zero status flags.
- Built a reusable testbench with automatic pass/fail checking.
- Applied directed, boundary, and randomized verification techniques.
- Learned to generate and inspect VCD simulation waveforms.
- Gained experience organizing a hardware-design project for GitHub.

11. Future Improvements

- Extend the ALU to 16-bit or 32-bit operation.
- Add comparison and shift operations.
- Refactor the design into smaller reusable RTL submodules.
- Perform synthesis and inspect the synthesized RTL/netlist.
- Explore FPGA implementation and hardware testing.

12. Conclusion

An 8-bit combinational ALU was successfully designed using Verilog HDL and functionally verified. The implementation supports eight arithmetic and bitwise operations and generates Carry and Zero status flags. The verification environment combined directed testing, edge-case testing, and 100 randomized tests. All 113 executed tests passed, demonstrating the expected functional behavior of the design under the tested conditions. The project provided practical experience in RTL coding, digital verification, simulation, waveform analysis, and project documentation.

13. Author

Dharshan R

Electronics and Communication Engineering
IIIT Kottayam